CS2106 Cheatsheet, AY26/27 S1

Zhao Wu and Tien Cheng

August 30, 2026

1 Introduction

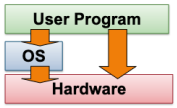
What is OS?

OS is a program that acts as an **intermediary** between a **computer user** and the **computer hardware**. (Simplified definition)

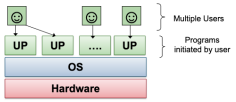
- E.g.
  - Windows, macOS, Linux, Solaris, FreeBSD
  - iOS, Android
  - PS5, Xbox, Nintendo Switch
  - Smart TV, Smart Watch

Brief History of OS

1. First computer:
   - **OS Type**: No OS
     - Program directly interacts with hardware.
     - Reprogram by physically changing the hardware configuration (cables, switches, punched paper tape)
   - **Advantage**:
     - Minimal overhead due to OS
   - **Disadvantage**:
     - Not portable (to port, need to manually rewrite the program)
     - Inefficient use of the computer
   - E.g. Electronic Numerical Integrator And Computer (ENIAC), Harvard Mark I
2. Mainframes
   - **Characteristics**:
     - No interactive interface (program via paper tape, magnetic tape, punch card)
     - Batch Processing Only
   - **OS Type**: Batch OS
     - User still interacts with hardware directly
     - Additional information for OS (e.g. resource required, job specification)
   - **Advantage**:
     - Execute user programs (i.e. jobs) one at a time
   - **Disadvantage**:
     - Simple batch processing is inefficient as the CPU is idle during I/O.
     - Multiprogramming is absent.
   - E.g. IBM 360



3. Time-Sharing OS
   - **OS Type**: Time-Sharing OS
   - **Advantage**:
     - Allow multiple users to interact with the machine using terminals (teletypes)
     - User job scheduling (illusion of concurrency)
     - OS manages CPU time, memory and storage
     - Virtualization of hardware (each program executes as if it has all the resources to itself)
   - E.g. Apple II PC, IBM PC



4. Personal OS
   - Windows model:
     - Single user at a time, but possibly more than 1 user can access
     - General time-sharing model
   - Unix model:
     - One user at the workstation, but other users can access remotely
     - General time-sharing model

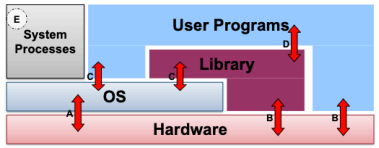
1.1 Motivation of OS

1. **Abstraction**
   - Hide low-level details and present higher-level functionality to the user
   - Motivation:
     - Large variation in hardware configuration, but the same hardware has well-defined functionality. (E.g. hard disk rotation speed varies, but it can store and retrieve information)
   - Efficiency, Programmability and Portability
2. **Resource Allocator**
   - Manage all resources (CPU, Memory, I/O) and arbitrate potentially conflicting requests for efficient and fair resource use
   - Motivation:
     - Program requires multiple hardware resources to run.
     - Multiple programs should run simultaneously for better utilisation
3. **Control Program**
   - Control execution of programs to:
     - Prevent errors and improper use of the computer
   - Motivation:
     - Program may "misuse" the computer
       - Bugs (accident), Virus, Malware (malicious)
     - Multiple users can share the computer
   - Security, isolation and protection

1.2 OS Structures

High-Level View of OS

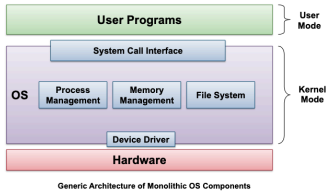
- OS is software that runs in **Kernel Mode** (i.e. direct access to all hardware resources)
  - Can't use system calls in kernel code
  - Can't use normal libraries and I/O
- Other software operates in **User Mode** (i.e. limited/controlled access to hardware resources)



- **A**: OS executing machine instructions
- **B**: Normal machine instructions executed (program/library code)
- **C**: Calling OS using ***system call interface***
- **D**: User program calls library code
- **E**: System processes
  - Provide high-level services, usually part of OS

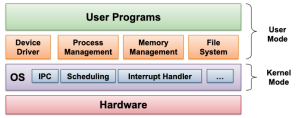
OS Structures

1. **Monolithic**
   - Big program (e.g. Linux source code)
   - If Kernel fails, BSOD
   - Better performance



Generic Architecture of Monolithic OS Components

2. **Microkernel**
   - Smaller and cleaner abstraction
   - Provides basic and essential facilities:
     - Inter-Process Communication
     - Address space management
     - Thread management
   - Higher-level OS services are run outside of the kernel, using IPC to communicate.
   - Lower performance

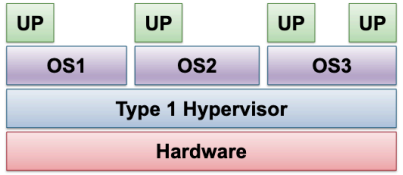


Generic Architecture of Microkernel OS Components

1.3 Virtual Machines

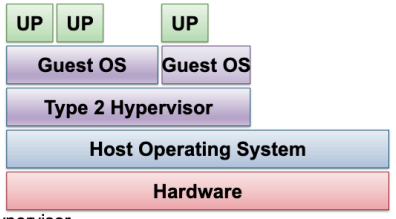
A **Virtual Machine** emulates the underlying hardware and is created and managed by a **Hypervisor** (aka **Virtual Machine Monitor (VMM)**)

1. Type 1 Hypervisor



- Type 1 hypervisor:
  - AKA bare-metal hypervisor
  - Provides individual *virtual machines* to guest OSes
  - E.g., IBM VM/370, VMware ESXi

2. Type 2 Hypervisor



- Type 2 hypervisor
  - Runs in host OS
  - Guest OS runs inside Virtual Machine
  - e.g., VMware Workstation, VirtualBox, QEMU

2 Process Abstraction

Motivation

- Allowing only one program to run at a time is inefficient, so the OS enables multiple programs to run concurrently.
- To switch between programs, the OS needs to save the context of the current program and load the context of the next program.
- A **process** (or task or job) is the OS abstraction for a running program.

2.1 Process Context

To allow multiple programs to run concurrently, the OS manages the following components:

Layer	Component	Stores
Memory	Text	program instructions
	Data	global variables and static variables
	Stack	collection of stack frames (calls + locals)
	Heap	region of memory used to store dynamically allocated data
Hardware	General-Purpose Registers (GPRs)	temporary data
	Program Counter (PC)	next instruction address
	Stack Pointer (SP)	top of stack frame address
	Frame Pointer (FP)	fixed location in stack frame address
OS	Process ID (PID)	unique process ID
	Process State	current process state

2.2 Stack Memory & Function Calls

Stack Memory

- The **stack** is a region of memory used to support function calls.
- Each function invocation **pushes a new stack frame onto the stack**, while each function return **pops the top stack frame**.
- Each stack frame contains (CS2106 convention, caller-pushed first):

Component	Who	Why
Local variables	Callee	Callee's own local variables
Parameters (only those that don't fit in registers)	**Caller**	so the callee can read the parameters of function call
Saved GPRs	Callee	Copy of GPRs which callee modifies; protect the caller's values across the call
Saved old SP	Callee	restore the caller's SP
Saved old FP	Callee	restore the caller's FP
Return address / Saved PC	**Caller**	return to the right instruction after the callee returns

- The stack pointer (SP) points to the top of the stack
- The frame pointer (FP) points to the base of the current stack frame
- We use a frame pointer because the stack pointer tracks the current top of the stack, which can move as local variables are pushed and popped. The FP is fixed, so we can use a fixed offset to access arguments and local variables.
- Restoring the SP does not erase the stack frame, so always initialise local variables as uninitialised local variables can contain leftover values from previous calls.

Function Call Convention

Function calling convention differs depending on the hardware and software. The following is an example of such convention in 2106:

1. **Function Call Preparation**:
   1. Caller: pass parameters using registers and/or the stack
   2. Caller: save return program counter onto the stack
2. **Transfer of Control from Caller to Callee**:
   1. Callee: **save registers used by the callee, save the old frame pointer**, save the old stack pointer
   2. Callee: allocate space for local variables of the callee on the stack
   3. Callee: adjust the stack pointer to point to the new stack top, **adjust the frame pointer**
3. **Callee Function Executes** (during that time, the callee may invoke other functions, but this can be abstracted away)
4. **Callee Function Returns**
   1. Callee: place the return result in a register (if applicable, usually a dedicated return register)

2. Callee: **restore saved registers and frame pointer**, restore the saved stack pointer

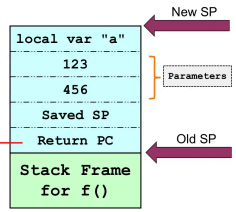
5. **Transfer of Control from Callee back to Caller using saved PC**:
   1. Caller: uses the return result (if applicable)
   2. Caller: continues execution of the program

```
void f(int a, int b)
{
    int c;

    a = 123;
    b = 456;
    c = g(a, b);
    ....
}

int g(int i, int j)
{
    int a;

    a = i + j
    return a * 2;
}
```



Register Spilling

- **Register Spilling**: When a function has more arguments than the number of registers, the extra arguments are spilled to the stack
- Even if neither function is short of registers, the caller and callee may use the same registers, so we need to save the registers and restore them after the call.
- There are two methods to save the registers:
  - **Callee-saved**: the callee saves the old values of the registers into its stack frame and restores them after the call
  - **Caller-saved**: the caller saves needed registers into its stack frame and restores them after the call
- CS2106 convention: all registers are callee-saved unless otherwise specified

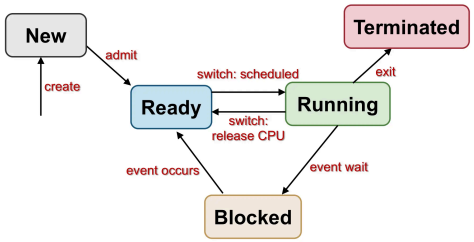
2.3 Heap Memory

Heap Memory

- The **heap** is a region of memory used to store dynamically allocated data.
- Dynamically allocated data in C: `malloc` and `free`
- Dynamically allocated data cannot be stored in:
  - Data region: size must be known at compile time
  - Stack: the data may outlast the function call
- Heap management is harder than stack management as allocation and deallocation can happen in arbitrary orders, leading to fragmentation.

2.4 Process States

Process State Model

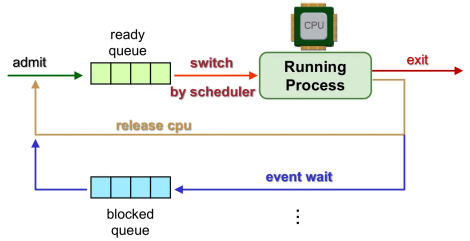


- A process can be in one of the following states:
  - **New**: process has been created but not fully initialised/admitted to the system
  - **Ready**: process is waiting to execute
  - **Running**: process is currently executing
  - **Blocked**: process is waiting for an event to occur (e.g. I/O completion, signal)
  - **Terminated**: process has finished execution

Multi-Process Management

- With one CPU core, at most one process can be running at a time
- With m CPU cores, at most m processes can be running at a time
- CS2106 convention: assume one CPU core unless otherwise specified
- Different processes may be in different states at the same time

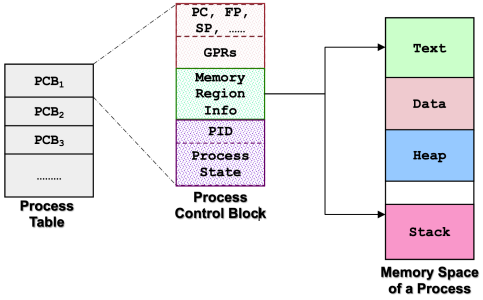
Process Queues



- The OS maintains a queue of processes for each state
  - **Ready Queue**: processes that are ready to be scheduled to run on the CPU
  - **Blocked Queue**: processes that are waiting for an event to occur
- More than one process can be in the ready and blocked queues at the same time
- There may be separate queues for different types of blocked processes (e.g. I/O blocked, signal blocked)

2.5 Process Control Block (PCB)

A **Process Control Block (PCB)** or **Process Table Entry** is a data structure that describes the execution context for a process, maintained by the Kernel.



2.6 System Calls

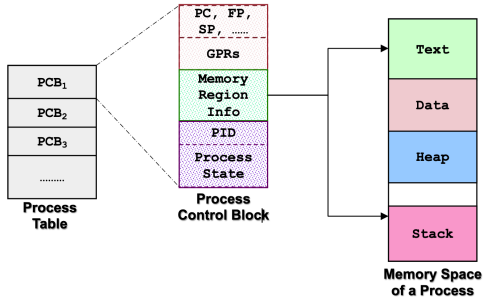
A **System Call** is an API to the OS that allows a user program to request services from the Kernel.

Difference in OS

- Unix Variant:
  - Follows POSIX standards
  - Small number of calls
- Windows Variant:
  - Uses `Win` API across different Windows versions
  - New version of windows add more calls.
  - Huge number of calls

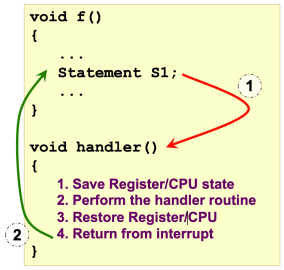
System Call Mechanism

1. User invokes the library call (e.g. `getpid()`)
2. Library call places the system call number into a register.
3. Library call invokes the **trap** instruction to switch from user mode to kernel mode.
4. In kernel mode, the **dispatcher** identifies the system call and passes control to the appropriate system call handler.
5. System call handler executes the system call
6. System call handler ended by restoring CPU state and return to user mode.
7. Library call returns the result to the user program.



Exception and Interrupt

- **Exception**:
  - Occurs during the execution of a program
  - Synchronous in nature (i.e. caused by the program itself)
  - E.g. divide by zero error or a page fault
- **Interrupt**:
  - Occurs during the execution of a program
  - Asynchronous in nature (i.e. caused by external events)
  - E.g. hardware interrupt or a software interrupt



1. Exception/Interrupt occurs:
   - Control transfer to a handler routine **automatically**
2. Return from handler routine:
   - Program execution resume
   - **May** behave as if nothing happened

2.7 A Case Study in Unix

Process Abstraction

Unix process management centres on `fork()`, `exec()`, `exit()`, and `wait()`.

In Unix, an entry in the PCB consists of:
1. Identification:
   - PID: Process ID (integer identifier)
2. Information:
   - Process State: Running, Sleeping/Suspended, Stopped, Zombie, etc.
   - Parent PID (PPID)
   - Cumulative CPU time
   - Other accounting and resource-management information

Use `ps` (process status) to inspect process information; `man ps` for options.

2.7.1 Process Creation

Unix vs Windows

- **Windows**: spawn a new process in (pretty much) a single syscall — path, arguments, etc.
- **Unix**: `fork()` clones the current process (new PID; continues from the instruction after `fork()`), then `exec()` replaces the image (discards old text/data/stack and execution state).

fork()

`int fork()` is the primary Unix mechanism for creating a process. It creates a child from the currently executing parent.

- Both parent and child continue from **immediately after** the `fork()` call.
- Child is initially an **almost exact duplicate**:
  - Same code and **initial** address-space **contents** (conceptually duplicated, **not** shared)
  - Copied register and execution context
  - Different PID and PPID
  - Different `fork()` return value: parent receives the child's PID; child receives `0`

```
printf("I am ONE\n");
fork(); // second process spawned; both continue from here
printf("I am seeing DOUBLE\n");
// I am ONE
// I am seeing DOUBLE
// I am seeing DOUBLE
```

Using fork()'s return value

Parent and child execute the same instructions after `fork()`. Use the return value to assign distinct work:

- `result != 0` — parent (can continue accepting work)
- `result == 0` — child (performs a separate task)

```
int result = fork();
if (result != 0) { // parent
  printf("P:My Id is %i\n", getpid());
  printf("P:Child Id is %i\n", result);
} else { // child
  printf("C:My Id is %i\n", getpid());
  printf("C:Parent Id is %i\n", getppid());
}
// P:My Id is 1234 / P:Child Id is 5678
// C:My Id is 5678 / C:Parent Id is 1234
```

Nondeterministic scheduling

- Parent/child order is **nondeterministic**: parent first, child first, or interleaved.
- On a single core they do not execute at the exact same instant, but the scheduler may alternate.

Independent Address Spaces

Parent and child initially **see equivalent values** after `fork()`, but do **not** share ordinary memory.

- Stack, heap, data, and code image are **conceptually duplicated**.
- Modifying a variable in one process does **not** modify the other.
- Open files and working directory are inherited (shared kernel resources), not the address space.
- If `var` starts as `1234`, parent can increment its copy while child decrements its own. Print order may vary; each process's private value changes independently.

2.7.2 exec()

exec() replaces the process image

By itself, `fork()` only duplicates the current program. `exec()` **replaces** the current process image:

- Replaces current code and data with a new executable
- Begins at the new program's entry point
- Discards the old stack and execution state
- **Retains the same PID** and broader process identity
- Variants: `execl`, `execv`, `execve`, `execlp`, `execvp`
- **Successful `exec()` does not return** — the old image no longer exists

Command-Line Arguments

```
int main(int argc, char *argv[])
```

- `argc`: number of arguments, **including** the program name
- `argv`: array of C strings; `argv[0]` is conventionally the executable name

execl()

`execl(path, arg0, ..., NULL)` supplies the executable path, the argument list, and a final `NULL`.

```
execl("/bin/ls", "ls", "-al", NULL); // replaces current program with ls -al
// NULL marks the end of the argument list
```

The fork() + exec() Pattern

Standard Unix pattern for launching a new program:
1. Parent calls `fork()` to create a child
2. Child calls `exec()` to run the requested executable
3. Parent remains available to continue its own work

Shells use this: shell forks; child execs the command; shell can later `wait()` for the child.

`init` and Process Tree

A process can only be created by forking an existing process, so Unix processes form a **process tree**.

- Root is **init**: created by the kernel during boot, traditionally **PID 1**
- Common ancestor of user processes; typically spawns OS/system programs
- Adopts orphaned processes
- Implementation differs by OS; on many Unix systems `init` is a symlink (e.g. to `systemd`)
- **Cannot be killed** even though it runs in user space; if it crashes → **kernel panic**

2.7.3 Process Termination

`exit()`

`exit(int status)` terminates the current process. **Does not return**.

- `exit(0)`: normal/successful termination
- Non-zero: error or abnormal condition
- Returning from `main()` **implicitly** invokes `exit()`; `main`'s return value becomes the exit status
- Open output streams are flushed; file descriptors are released
- On process `exit()`:
  - Process state is set to **Zombie**
  - Most resources are released (e.g. file descriptors)
  - Some information is **not releasable** (so the parent can `wait()`):
    - PID
    - Exit status
    - Process accounting (e.g. CPU time)

`wait()`

`wait(int *status)` lets a parent synchronise with a child:

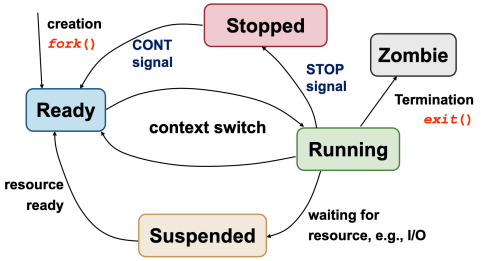
- **Blocks** until at least one child terminates
- Returns the PID of a terminated child
- Stores the child's exit status through `status`, unless `NULL`
- Kernel can write into the parent's memory because it is privileged
- Variants: `waitpid()` — wait for a specific child; `waitid()` — wait for child state changes
- Several children ⟹ several `wait()` calls to reap them all

Zombie vs Orphan

- **Zombie**: child that has **exited** but has not yet been **reaped** by its parent via `wait()`
  - Cannot execute or be meaningfully killed (already dead)
  - Remaining PCB occupies a process-table entry; too many can exhaust the table (older Unix: may need reboot)
- **Orphan**: a **still-running** child whose parent has terminated
  - `init` becomes its pseudo-parent
  - When the adopted child later terminates, `init` `wait()`s and cleans up
  - If the parent dies while the child is **already a zombie**, `init` reaps that leftover state too

Parent–Child Lifetime

1. Parent forks a child
2. Child optionally execs a new program
3. Child exits and becomes a zombie
4. Parent waits
5. Kernel removes the child's remaining process-table entry



Unix Process States

Running, Sleeping/Suspended, Stopped, Zombie. Major transitions:

- `fork()` creates a **ready** process
- Context switch: ready → running
- Running waiting for a resource (e.g. I/O) → suspended
- Resource ready → suspended → ready
- Stop/continue signals move between running/stopped and ready
- `exit()` → zombie, then final cleanup after `wait()`

2.7.4 Implementing `fork()`

Simplified `fork()` steps

1. Create the child address space
2. Allocate a new PID
3. Create kernel process data structures / PCB entry
4. Copy relevant kernel environment (e.g. scheduling priority)
5. Initialise child PID, PPID, and CPU accounting
6. Copy program, data, heap, and stack
7. Acquire shared system resources (open files, working directory)
8. Initialise child hardware context by copying registers
9. Place the child in the scheduler's ready queue

A literal full memory copy is expensive (entire address space).

Copy-on-Write (COW)

- Parent and child initially **share** memory **pages**
- Reads continue sharing
- A page is duplicated only when one process **writes** to it
- Memory is organised into **pages** (consecutive ranges of locations) and managed at **page** granularity, not per byte

`clone()`

Modern Linux provides `clone()` for partial duplication / selected resource sharing, instead of a full `fork()`-style copy.

Key Unix Process System Calls

- `fork()` — create a child
- `exec()` family — replace the current image
- `exit()` — terminate and report status
- `wait()` family — synchronise and collect termination status
- `getpid()` / `getppid()` — current and parent PID